# STM32基础 - 定时器中断与输出比较

## 课程目标

1. 熟悉TIM定时器，掌握定时中断原理
2. 掌握输出比较和PWM输出控制
3. 掌握对射式传感器、OLED、直流电机等基本元器件的控制

## 课程试验

1. 利用STM32完成定时器定时中断试验
2. 利用STM32完成PWM驱动LED呼吸灯试验
3. 利用STM32完成PWM驱动直流电机试验

## 课堂引入

在我们的生活中，时间是一种非常重要的资源，它可以帮助我们规划和管理任务，跟踪时间流逝，以及测量各种物理参数。在微控制器领域，定时器是一种常见的功能，它可以帮助我们精确地控制时间，执行各种任务。STM32系列微控制器是一款广泛应用的微控制器，其内部集成了丰富的定时器资源。这些定时器可以帮助我们完成各种应用，例如定时中断（事件触发）、时间测量、PWM控制等。在本章节中，我们将学习如何使用STM32的定时器，并了解其基本原理和配置方法。

## 授课进程

### 一、定时中断

#### 1、基本介绍

TIM（Timer）定时器，可以对输入的时钟进行计数，并在计数值达到设定值时触发中断

- 时基单元
    - 16 位计数器（数多少，可以是 向上数，向下数，中间对齐数） 计数器寄存器 (TIMx_CNT)
    - 16 位预分频器（分频多少） 预分频器寄存器 (TIMx_PSC)
    - 自动重装寄存器（数到多少就停） 自动装载寄存器 (TIMx_ARR)
- 72MHz计数时钟下可以实现最大59.65s的定时？
    - 72000000/65536/65536 取倒数
    - 如果想要记录更大的定时怎么办？ 级联另一个定时器，即可*65536 *65536

作用：

- 定时中断功能

- 内外时钟源选择、输入捕获、输出比较、编码器接口、主从触发模式等多种功能

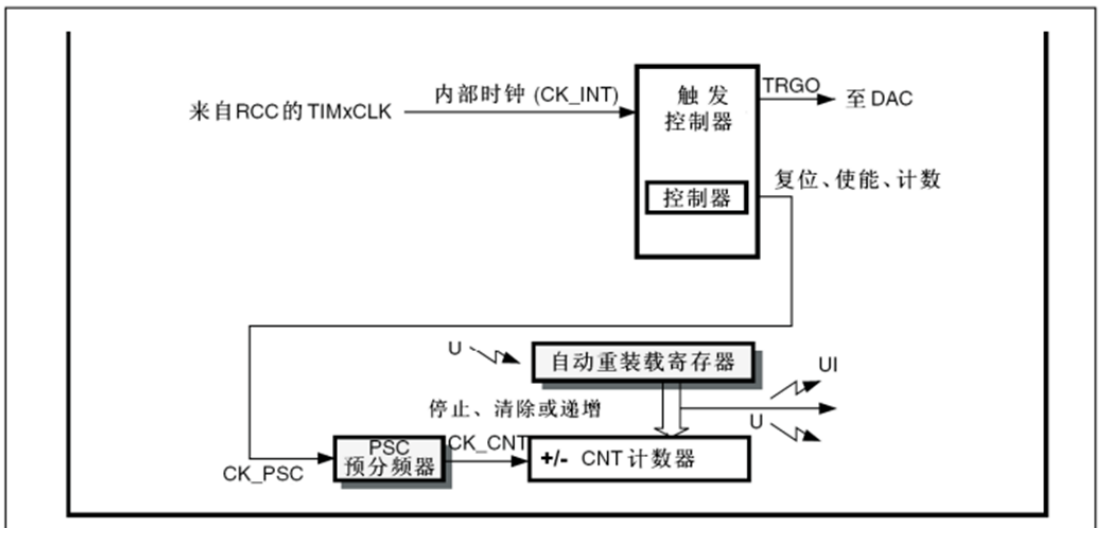
**2、定时器类型**

| 类型 | 编号 | 总线 | 功能 |
| --- | --- | --- | --- |
| 高级定时器 | TIM1、TIM8 | APB2 | 拥有通用定时器全部功能，并额外具有重复计数器、死区生成、互补输出、刹车输入等功能 |
| 通用定时器 | TIM2、TIM3、TIM4、TIM5 | APB1 | 拥有基本定时器全部功能，并额外具有内外时钟源选择、输入捕获、输出比较、编码器接口、主从触发模式等功能 |
| 基本定时器 | TIM6、TIM7 | APB1 | 拥有定时中断、主模式触发DAC的功能 |

计划使用的STM32F103C8T6芯片拥有的定时器资源只有**TIM1、TIM2、TIM3、TIM4**，也就是一个高级定时器，三个通用定时器，没有基本定时器。

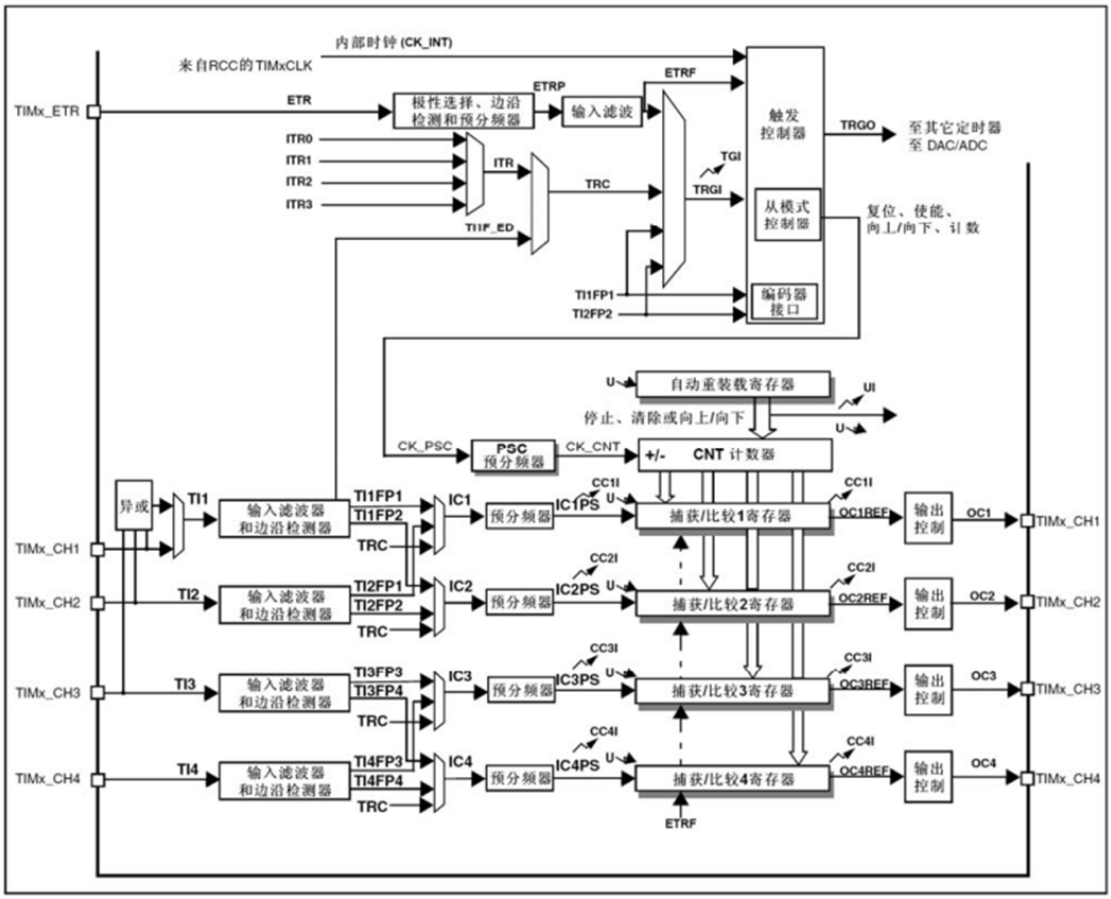
（1）基本定时器

图144　基本定时器框图



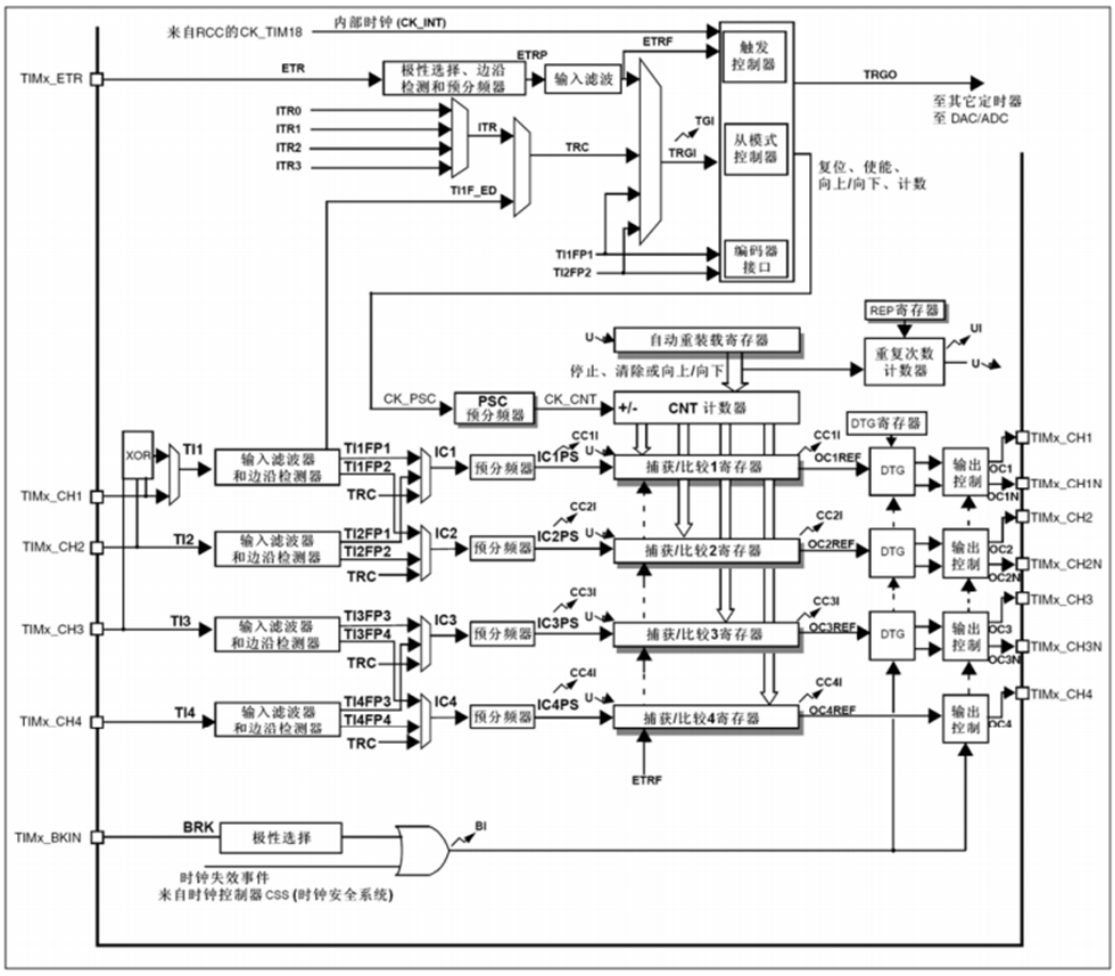

（2）通用定时器

图98　通用定时器框图



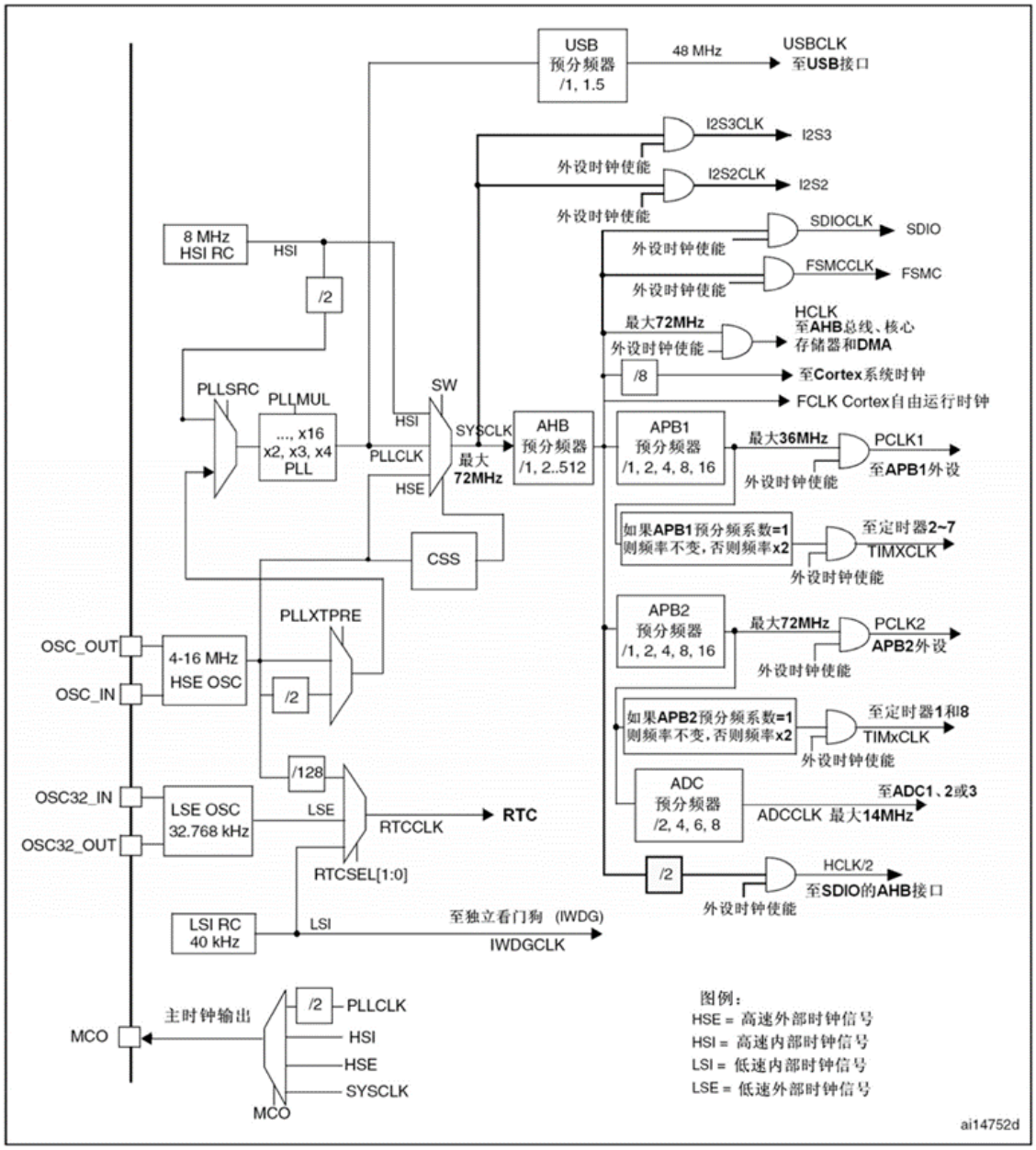

（3）高级定时器

图50　高级控制定时器框图

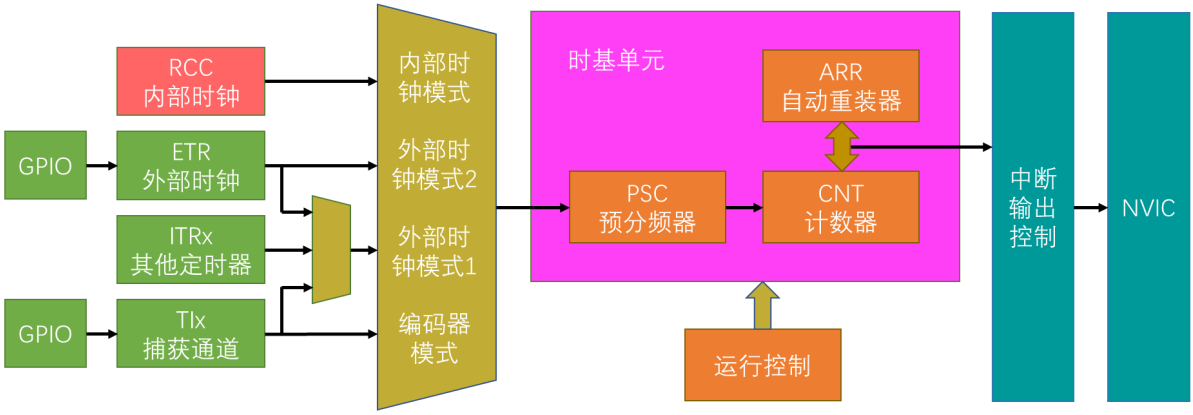


## 3、RCC时钟树

图8　　时钟树



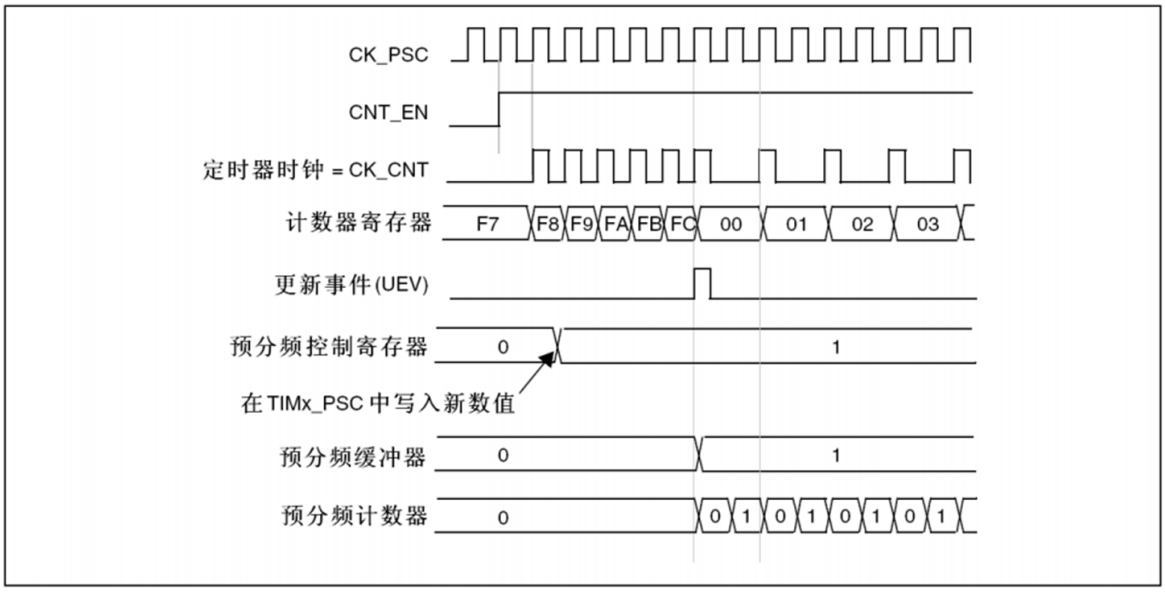

## 二、定时中断基本结构



这是定时中断基本结构图，在这个图里额外加入了定时中断和内外时钟源，中间最重要的是由PSC预分频器、CNT计数器、ARR自动重装计数器三个寄存器构成的时基单元。其下的运行控制主要控制寄存器的位，比如启动停止、向上或向下计数等。操作这些寄存器就可以控制时基单元单元的运行。左边是为时基单元提供时钟，可以选择RCC提供的内部时钟，也可以选择ETR引脚提供的外部时钟模式2，还可以选择触发输入当做外部时钟，也就是外部时钟模式1。对应的有ETR外部时钟、ITRx其他定时器、TIx输入捕获通道。这些就是定时器的所有可选的时钟。最下方还有一个编码器模式，这一般是编码器独用的模式，普通的时钟用不到。接下来看右边，就是计时时间到产生更新中断后的信号去向。如果采用的是高级定时器的，这里还会多一个重复计数器。通用定时器就不用考虑了。中断信号首先会先在状态计数器里置一个中断标志位，标志位会通过中断输出控制到NVIC申请中断。为什么会有一个中断输出控制呢？因为定时器模块有很多地方都要申请中断。比如左边图示不仅更新要申请中断，触发信号也会申请中断，还有输入捕获和输出比较匹配时也会申请，所以这些中断都要经过中断输出控制。如果需要这个中断，那就允许；如果不需要，那就禁止。简单来说，这个中断输出控制，就是一个中断输出的允许位，如果需要某个中断，就允许一下就可以了。

1、预分频器

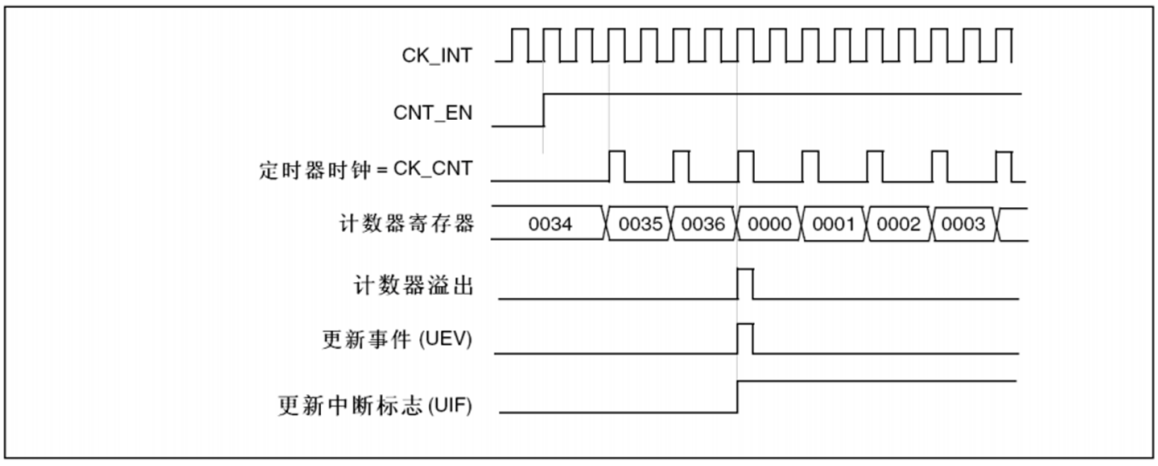
图99　　当预分频器的参数从1变到2时，计数器的时序图




计数器计数频率：CK_CNT = CK_PSC / (PSC + 1)

2、计数器

图102　计数器时序图，内部时钟分频因子为2




计数器溢出频率：CK_CNT_OV = CK_CNT / (ARR + 1)

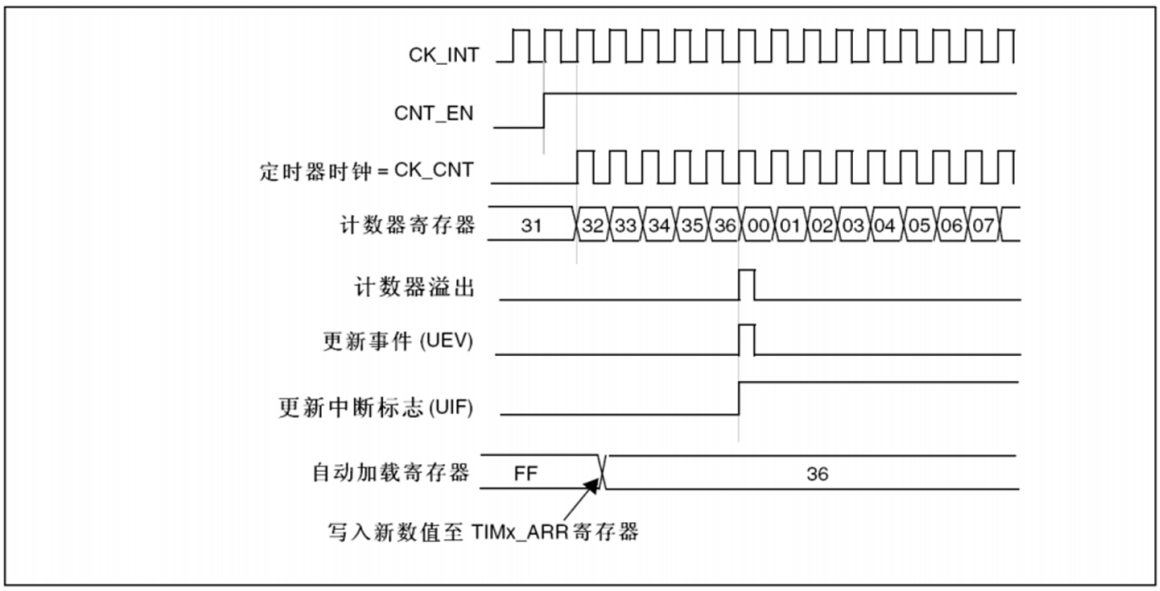
ARR （Auto Reload Register 自动重装寄存器）

代入预分频器的计数器计数频率公式可以得到下面的式子：

CK_CNT_OV = CK_PSC / (PSC + 1) / (ARR + 1)

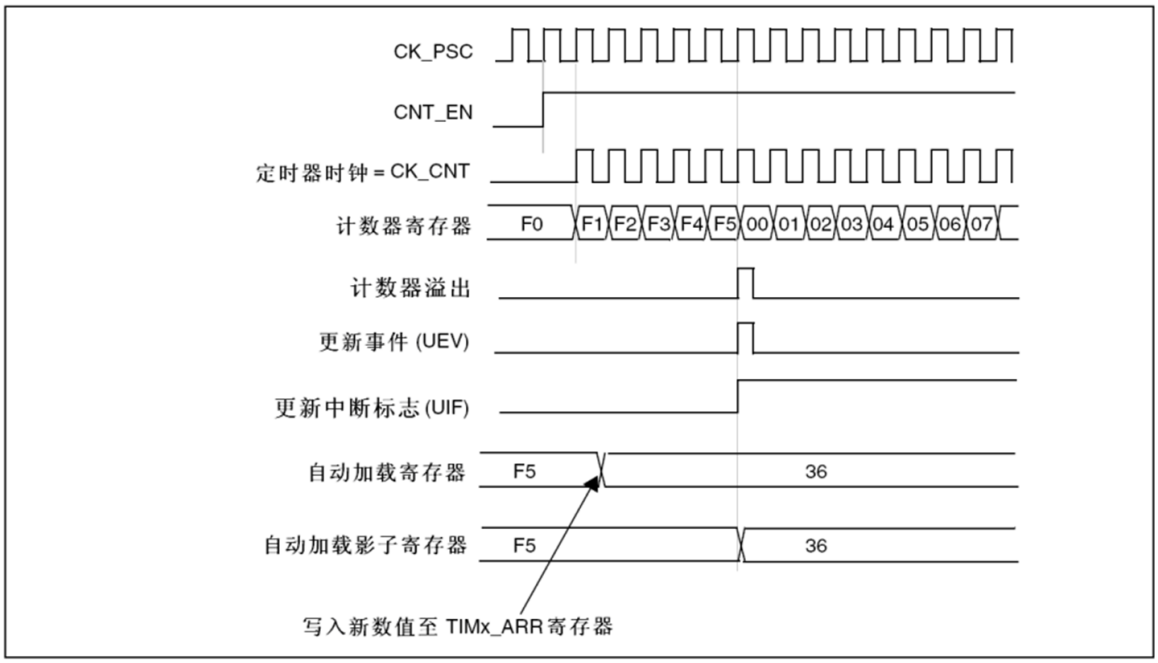
（1）计数器无预装时序

图105　计数器时序图，当ARPE=0时的更新事件(TIMx_ARR没有预装入)




（2）计数器有预装时序

图106　计数器时序图，当ARPE=1时的更新事件(预装入了TIMx_ARR)

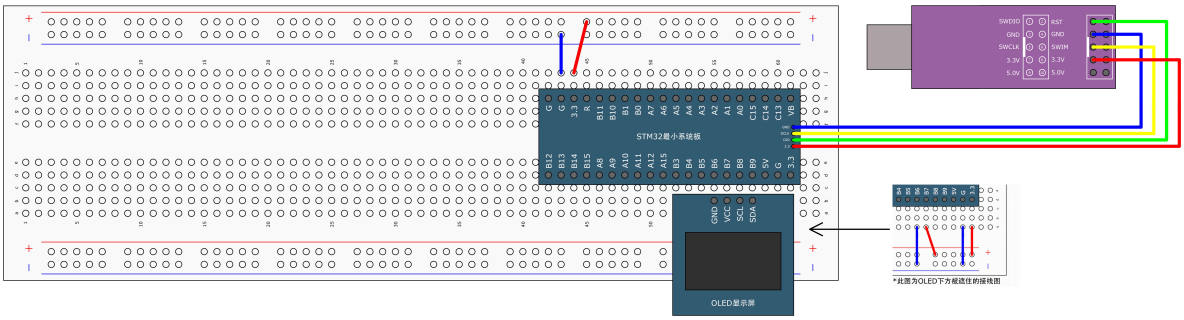



## 三、定时中断实验

（1）实验目的

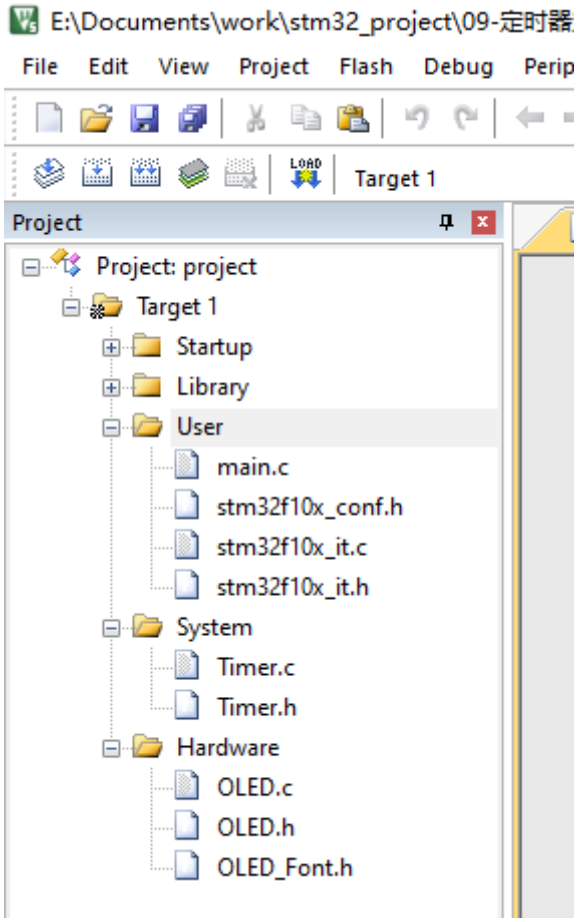
通过对STM32内部的定时中断实现中断计数，并在OLED上显示出计数信息。

（2）硬件电路接线图



（3）程序设计

工程结构如下图：




其中System组与之前工程不一样，这个组主要提供了定时器中断处理方法，具体代码展示如下：

Timer.c

```
#include "stm32f10x.h"                  // Device header

void Timer_Init(void) {
    // 设置时钟
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM2, ENABLE);

    // 设置使用内部定时器
    TIM_InternalClockConfig(TIM2);

    TIM_TimeBaseInitTypeDef TIM_TimeBaseInitStructure;
    TIM_TimeBaseInitStructure.TIM_ClockDivision = TIM_CKD_DIV1;     // 设置选1分频，不需要
分频
```

```
    TIM_TimeBaseInitStructure.TIM_CounterMode = TIM_CounterMode_Up;     // 设置计数器向上计数模式
    // 设定一个1秒的定时，就是定时频率为1Hz，可以参考 定时频率 = 72M/(PSC+1)/(ARR+1) 计算
    // ARR和PSC的取值都应在0~65535范围内
    // Period就是ARR自动重装器的值
    TIM_TimeBaseInitStructure.TIM_Period = 10000 - 1;
    // Prescaler就是PSC预分频器
    TIM_TimeBaseInitStructure.TIM_Prescaler = 7200 - 1;
    // RepetitionCounter是重复计数器，只有高级定时器才需要设置
    TIM_TimeBaseInitStructure.TIM_RepetitionCounter = 0;
    TIM_TimeBaseInit(TIM2, &TIM_TimeBaseInitStructure);     // 初始化时基单元

    // 手动清除更新中断标记位，避免刚上电初始化完就进中断的问题
    TIM_ClearFlag(TIM2, TIM_FLAG_Update);
    // 更新中断到NVIC的通路
    TIM_ITConfig(TIM2, TIM_IT_Update, ENABLE);

    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);
    NVIC_InitTypeDef NVIC_InitStructure;
    NVIC_InitStructure.NVIC_IRQChannel = TIM2_IRQn;
    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 2;
    NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
    NVIC_Init(&NVIC_InitStructure);     // 初始化NVIC

    // 使能定时器
    TIM_Cmd(TIM2, ENABLE);
}
```

Timer.h

```
#ifndef __TIMER_H
#define __TIMER_H

void Timer_Init(void);

#endif
```

main.c

```
#include "stm32f10x.h"                  // Device header
#include "OLED.h"
#include "Timer.h"

uint16_t Num;

int main(void){
```

```
    OLED_Init();
    Timer_Init();

    OLED_ShowString(1, 1, "Num:");

    while(1){
        OLED_ShowNum(1, 5, Num, 5);      // 显示计数器的值，也就是定时中断的次数
    }
}

// 处理定时中断，收到中断计数器Num自加
void TIM2_IRQHandler(void) {
    if (TIM_GetITStatus(TIM2, TIM_IT_Update) == SET) {
        Num++;
        TIM_ClearITPendingBit(TIM2, TIM_IT_Update);
    }
}
```

（4）代码完成后，编译并下载到硬件中执行，会发现实验结果是随着时间中断的到来OLED显示的计数信息正确，符合预期设计。

## 四、输出比较结果 OC

### 1、基本介绍

OC（Output Compare）输出比较。输出比较可以通过比较CNT与CCR寄存器值的关系，来对输出电平进行置1、置0或翻转的操作，用于输出一定频率和占空比的PWM波形。这里CNT就是时基单元里面的计数器，CCR就是捕获/比较寄存器，它是输入捕获和输出比较共用的。当使用输入捕获时，它就是捕获寄存器；当使用输出比较时，它就是比较寄存器。在输出比较时电路会比较CNT和CCR的值，CNT计数自增，CCR是预先给定的一个值，当CNT大于CCR，小于CCR或者等于CCR时，输出就会对应的置1或置0，这样就可以输出一个电平不断跳变的pwm波形了。每个高级定时器和通用定时器都拥有4个输出比较通道，其中高级定时器的前3个通道额外拥有死区生成和互补输出的功能。基本定时器是最简单的，没有输入捕获和输出比较的功能。

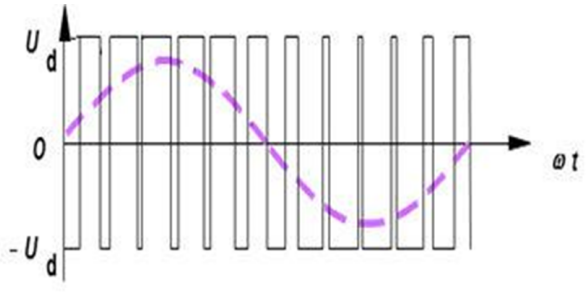
### 2、PWM简介

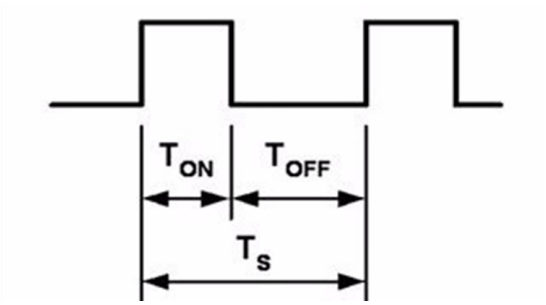
PWM（Pulse Width Modulation）脉冲宽度调制。PWM波形是一个数字输出信号，也是由高低电平组成的。

在具有**惯性的系统**中，可以通过对一系列脉冲的宽度进行调制，来等效地获得所需要的模拟参量，常应用于电机控速等领域。

PWM参数：

- 周期T（**重复波形发生的最小间隔**）
- 频率 = 1 / T （**1秒内出现了多少个周期**）
- 占空比 = Ton / Ts （**高电平占整个周期的比例**）
- 分辨率 = 占空比变化步距（**PWM最小能设定到的高电平时间所占周期的比例，也即最小占空比**）

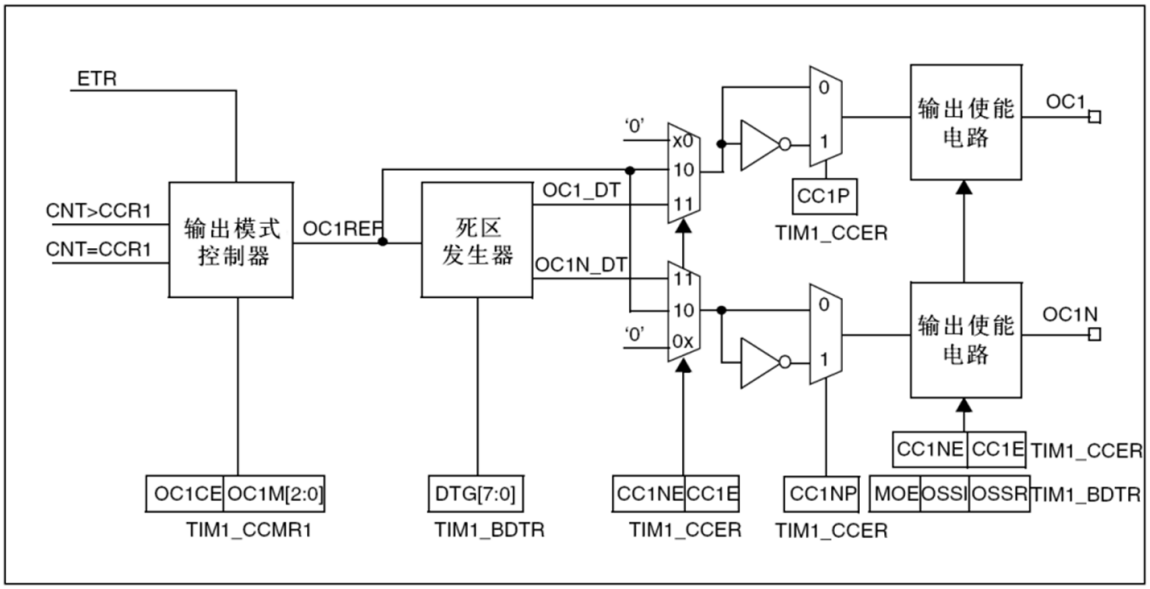



**3、输出比较通道**

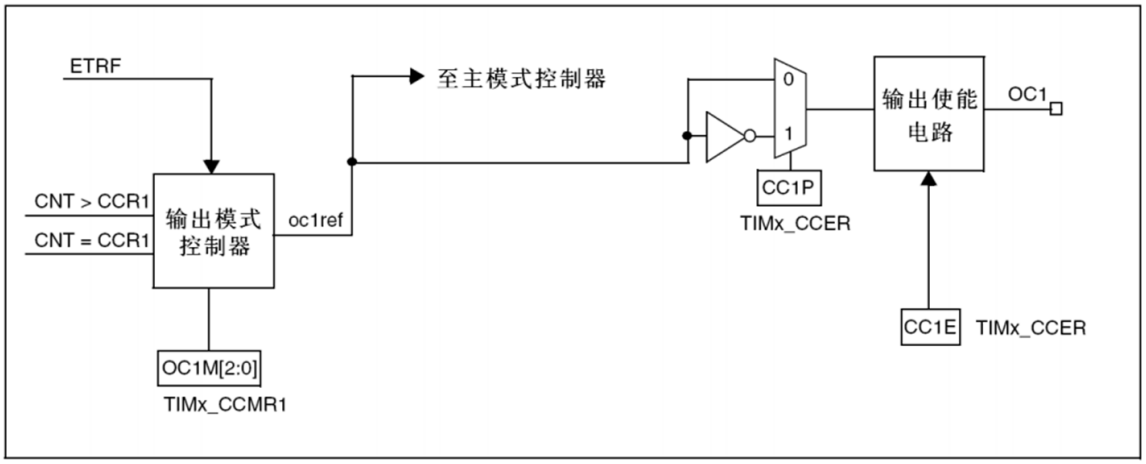
（1）高级

图78　捕获/比较通道的输出部分(通道1至3)




（2）通用

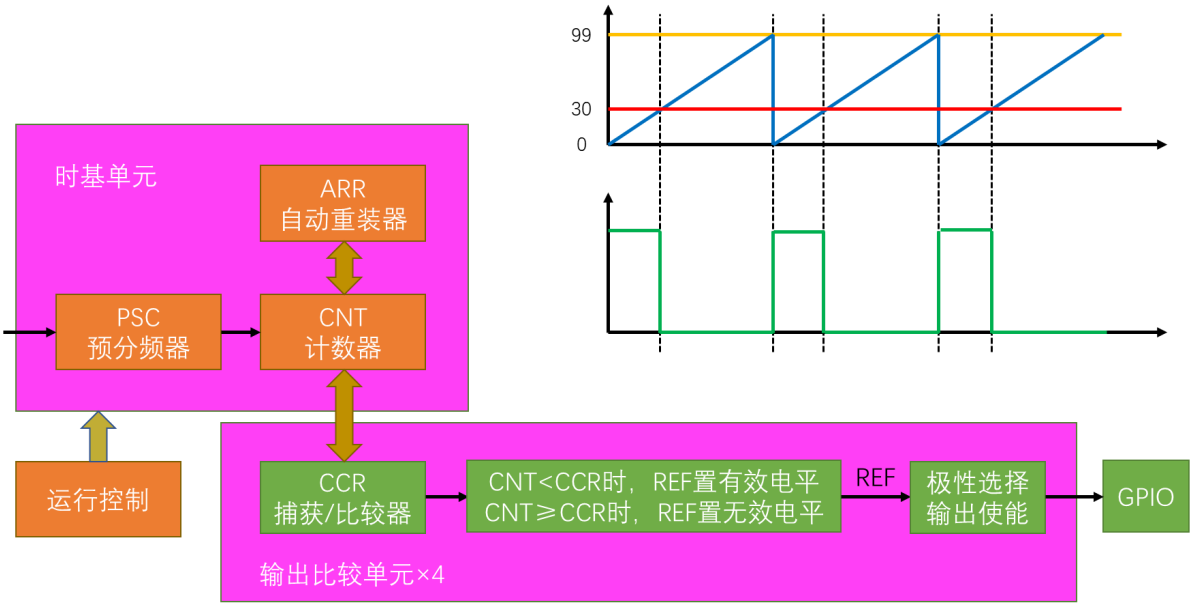
图125　捕获/比较通道的输出部分(通道1)




**4、输出比较模式 (CCR- Capture Compare Register)**

| 模式 | 描述 |
| --- | --- |
| 冻结 | CNT=CCR时，REF保持为原状态 |
| 匹配时置有效电平 | CNT=CCR时，REF置有效电平 |
| 匹配时置无效电平 | CNT=CCR时，REF置无效电平 |
| 匹配时电平翻转 | CNT=CCR时，REF电平翻转 |
| 强制为无效电平 | CNT与CCR无效，REF强制为无效电平 |
| 强制为有效电平 | CNT与CCR无效，REF强制为有效电平 |
| PWM模式1 | 向上计数：CNT<CCR时，REF置有效电平，CNT≥CCR时，REF置无效电平<br>向下计数：CNT>CCR时，REF置无效电平，CNT≤CCR时，REF置有效电平 |
| PWM模式2 | 向上计数：CNT<CCR时，REF置无效电平，CNT≥CCR时，REF置有效电平<br>向下计数：CNT>CCR时，REF置有效电平，CNT≤CCR时，REF置无效电平 |

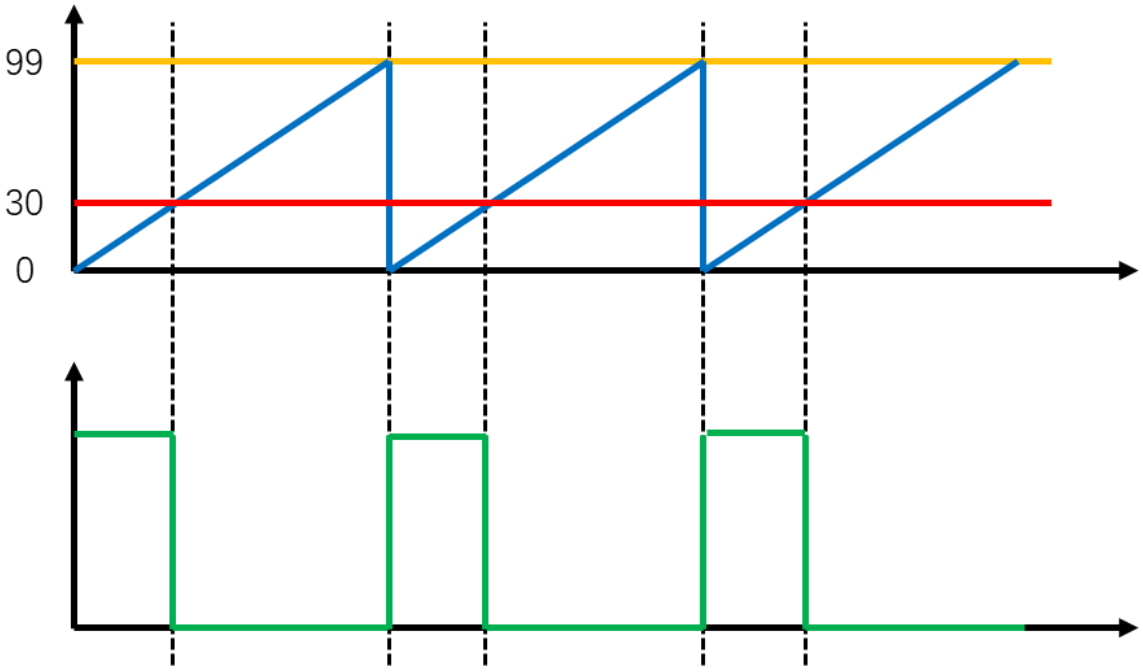
## 5、PWM基本结构




CCR： Capture Compare Register

左上角是时基单元和运行控制部分，在左边还有时钟源选择，因为是之前讲过的内容，所以上图省略了。在这里还需要继续使用，只不过是这里更新事件的中断申请不需要了，输出PWM暂时还不需要中断。配置好时基单元的CNT，就可以开始不断的自增运行了。然后在下面的输出比较单元总共有4路。输出比较单元最开始是CCR捕获/比较寄存器，CCR是自己设置的，CNT不断自增运行。它们两个不断进行比较，在后面的输出模式控制器中CNT小于CCR，就输出高电平；之后CNT大于等于CCR了，就变为低电平，当CNT溢出清零后，CNT又小于CCR，所以置高电频，再之后，就又使CNT大于等于CCR置低电平。这样一直持续下去，REF的电平就会不断变化，并且它的占空比是受CCR值的调控的。如果CCR设置高一些，输出的占空比就变大；CCR设置低一些，输出的占空比就变小。这就是PWM模式的工作流程，这里REF就是一个频率可调，占空比也可调的PWM模型，最终再经过极性选择，输出使能最终通向GPIO口，这样就能完成PWM波形的输出了。

## 6、参数的计算




PWM频率： Freq = CK_PSC / (PSC + 1) / (ARR + 1)

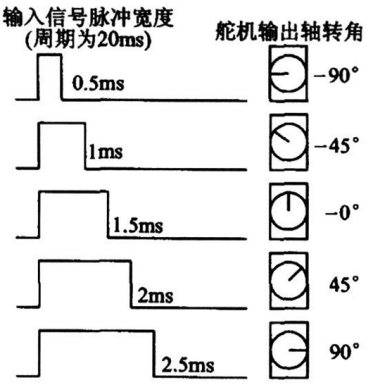
PWM占空比： Duty = CCR / (ARR + 1)

PWM分辨率： Reso = 1 / (ARR + 1)

## 五、电机

### 1、舵机

舵机是一种根据输入PWM信号占空比来控制输出角度的装置。



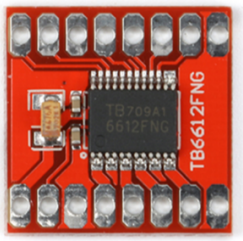

图中舵机的型号是SG90。它有三根输出线，两根是电源线，一根是信号线，PWM就是输入的这个信号线来控制舵机的。在图中左边有一个白色的输出轴，它的轴会固定在一个指定的角度不动，至于固定在哪个位置是由信号线的PWM信号来决定的，这就是舵机的工作方式。舵机的的执行逻辑是PWM信号输入到控制板，给控制板一个指定的目标角度。然后由定位器检测输出轴的当前角度，如果大于目标角度，电机就会反转；如果小于目标角度，电机就会正转，最终使输出轴固定在指定角度。

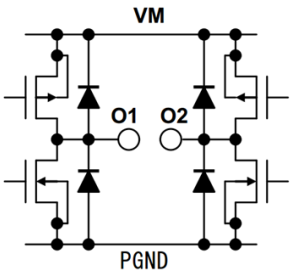
输入PWM信号要求：周期为20ms，对应的就是五十赫兹，高电平宽度为0.5ms~2.5ms，就是占空比是在这个范围对应的输出角度。

实际应用方面比如机器人、机械臂可以用舵机来控制关节，遥控车、遥控船可以用舵机来控制方向。

### 2、直流电机

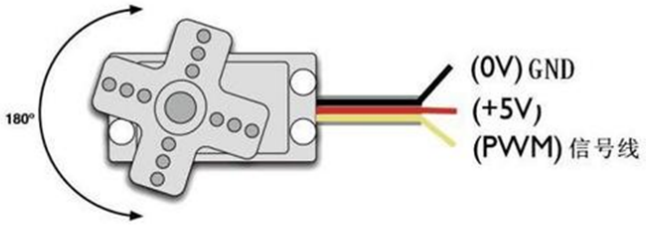
直流电机是一种将电能转换为机械能的装置，有两个电极，当电极正接时，电机正转，当电极反接时，电机反转。直流电机属于大功率器件，GPIO口无法直接驱动，需要配合电机驱动电路来操作。电机这类器件基本上都属于大功率设备，必须要加驱动电路才能控制。这里用TB6612驱动电路，它是一款双路H桥型的直流电机驱动芯片，可以驱动两个直流电机并且控制其转速和方向。这个芯片里面是有两路驱动电路的，可以独立的控制两个电机。又因为它是H桥型的驱动电路，里面一路有四个开关管，所以就可以控制正反转。

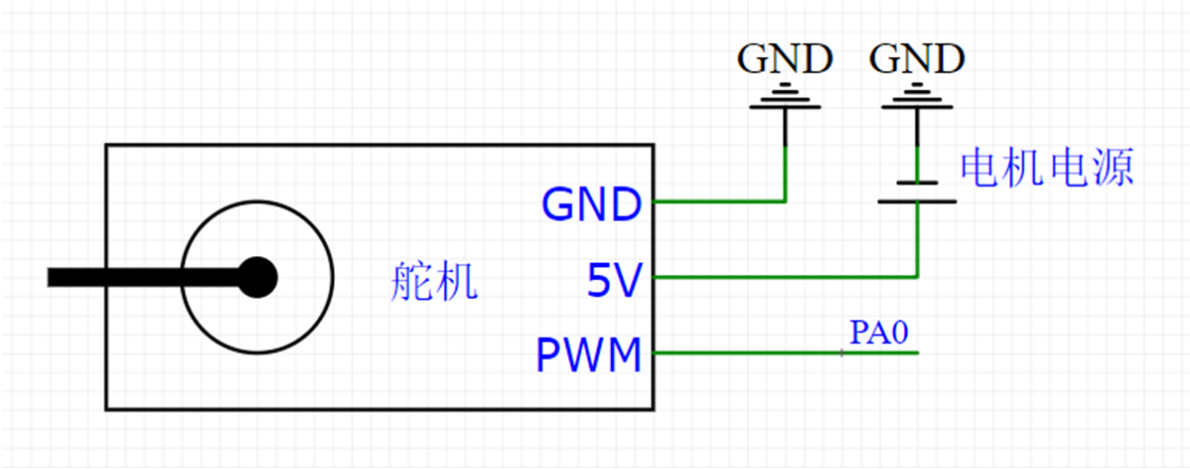



## 六、基本电路

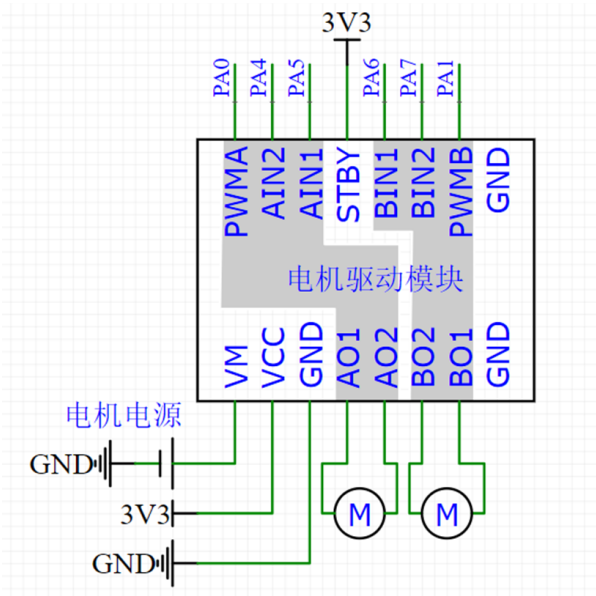
### 1、舵机




舵机的硬件电路是比较简单的，先看一下上面这个引脚定义图。在舵机上有三根线，分别是黑红黄，对应的黑色是电源负极GND，红色是电源正极，这里5伏舵机就接正5伏，黄色是信号线，接PWM信号。有些舵机的线颜色不太一样，可能是棕红橙，这种棕色是电源负极，红色是电源正极，橙色是信号线，引脚顺序是一样的。一般电机都是大功率设备，它的驱动电源也必须是一个大功率的输出设备。
如果能单独提供供电，那就再好不过了。如果不能，那就要注意电源的功率是不是能达标。单独供电的负极要和STM32共地，然后正极接在5伏供电引脚上，对于套件的话，可以直接从CMSIS-DAP的5伏输出角拧一根线接到这里，就是使用USB的5伏供电，这样也是可以带的动的。最后就是信号线PWM就直接接到STM32引脚上就行，比如PA0。因为这个舵机内部是有驱动电路的，所以单片机的引脚就可以直接接到这里的，PWM只是一个通信线是不需要大功率的。

**2、直流电机**




| | 引脚 | 定义 |
|---|---|---|
| | VM | 驱动电压输入端（4.5~10V） |
| | VCC | 逻辑电平输入端（2.7~5.5V） |
| | GND | 电源地端 |
| | STBY | 正常工作/待机状态控制输入端 |
| 1路电机 | PWMA | PWM信号输入端 |
| | AIN1 | 电机控制模式输入端 |
| | AIN2 | |
| | A01 | 电机驱动输出端 |
| | A02 | |
| 2路电机 | PWMB | PWM信号输入端 |
| | BIN1 | 电机控制模式输入端 |
| | BIN2 | |
| | B01 | 电机驱动输出端 |
| | B02 | |

| 输入 | | | | 输出 | | |
|---|---|---|---|---|---|---|
| IN1 | IN2 | PWM | STBY | O1 | O2 | 模式状态 |
| H | H | H/L | H | L | L | 制动 |
| L | H | H | H | L | H | 反转 |
| L | H | L | H | L | L | 制动 |
| H | L | H | H | H | L | 正转 |
| H | L | L | H | L | L | 制动 |
| L | L | H | H | OFF | | 停止 |
| H/L | H/L | H/L | L | OFF | | 待机 |

左下角VM是电机电源的正极，要接一个可以输出大电流的电源。通过右边的引脚定义可以看出VM的驱动电压输入端范围是4.5-10伏
这个电压一般要和电机额定电压保持一致，比如如果是5伏的电机，这里就接5伏；如果是7.2伏的电机，这里就接7.2伏。下一个引脚是VCC，这个是逻辑电平输入端，范围是2.7-5.5伏，这个要和STM32控制

器的电源保持一致接3.3伏。这个硬件并不需要大功率，所以可以和控制器共用一个电源。然后第三个引脚是GND，接系统的负极。然后板子右边还有两个GND，都是一样的，在板子内部都是连通的，随便选一个GND用就可以。AO1、AO2和BO1、BO2是两路电机的输出，可以像图中所示分别接两个电机。AO1和AO2就是A路的两个输出，它的控制端就是上面的三个引脚PWMA、AIN2和AIN1。这三个引脚控制下面A路的一个电机。这三个引脚就直接接到单片机的GPIO口就行了，其中PWMA引脚要接PWM信号输出端，其他两个引脚可以任意接两个普通的GPIO口。这三个引脚给一个低功率的控制信号，驱动电路就会从VM汲取电流来输出到电机，这样就能完成低功率的控制信号控制大功率设备的目的了。右边这一路也是一样的，BO1和BO2是闭路的两个输出，它的控制端是上面右边的BO1、BO2和PWMB这三个引脚，两路的功能和操作方法是完全一样的。最后还剩一个STBY引脚是待机控制脚，如果接GND，芯片就不工作处于待机状态；如果接逻辑电源，VCC芯片就正常工作。这个引脚如果不需要待机模式的话，可以直接接VCC 3.3伏，如果需要的话，可以任意接一个GPIO，给高低电平就可以控制了。控制电机正的反转和速度，看右下角的表，输入是IN1、IN2、PWM和STBY，STBY低电平就待机，高电平就正常工作。然后右边是输出O1、O2和模式状态。如果IN1和IN2全都接高电平，两个输出就都为低电平，这样两个输出没有电压差，电机是不会转的。如果IN1和IN2全都接低电平，输出直接关闭，这样电机也是不会转的。如果IN1给低电平，IN2给高电平，电机就处于反转状态。如果PWM给高电平，那输出就是一低一高，有电压差了电机可以转；如果PWM给低电平，那输出两个低电平，电机就是不转。如果PWM是一个不断翻转的电平信号，那电机不就是快速的反转停止，如果PWM频率足够快，那电机就可以连续稳定的反转了，并且速度取决于PWM信号的占空比，这就是反转的工作流程。在这里的PWM就是我们之前讲的，使用PWM来等效一个模拟量的功能。然后看下面正转也是差不多，这里给IN1高电平，IN2低电平，电机就是正转状态。转还是不转则要取决于PWM信号，PWM高正转，低停止，如果PWM频率足够快，那电机就是连续稳定的正转了，并且速度取决于PWM信号的占空比。

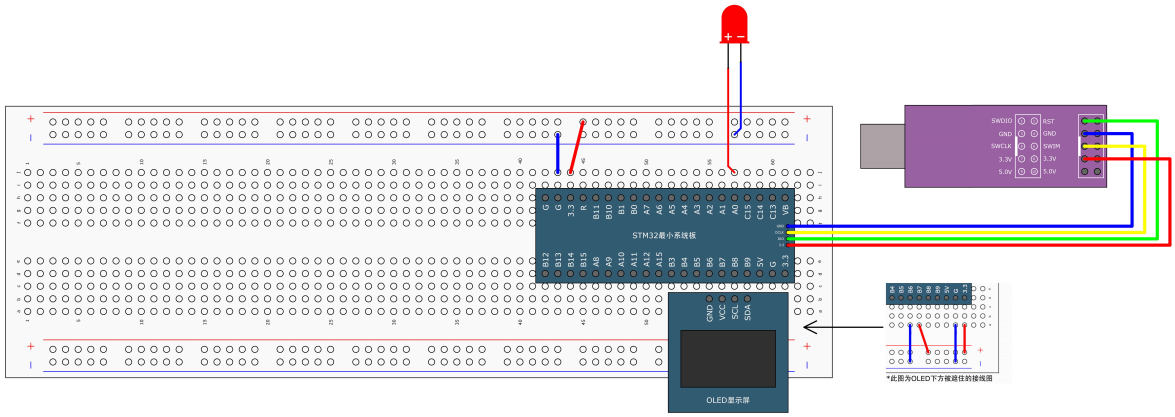
## 七、输出比较实验

### 1、输出PWM波形

（1）实验目的

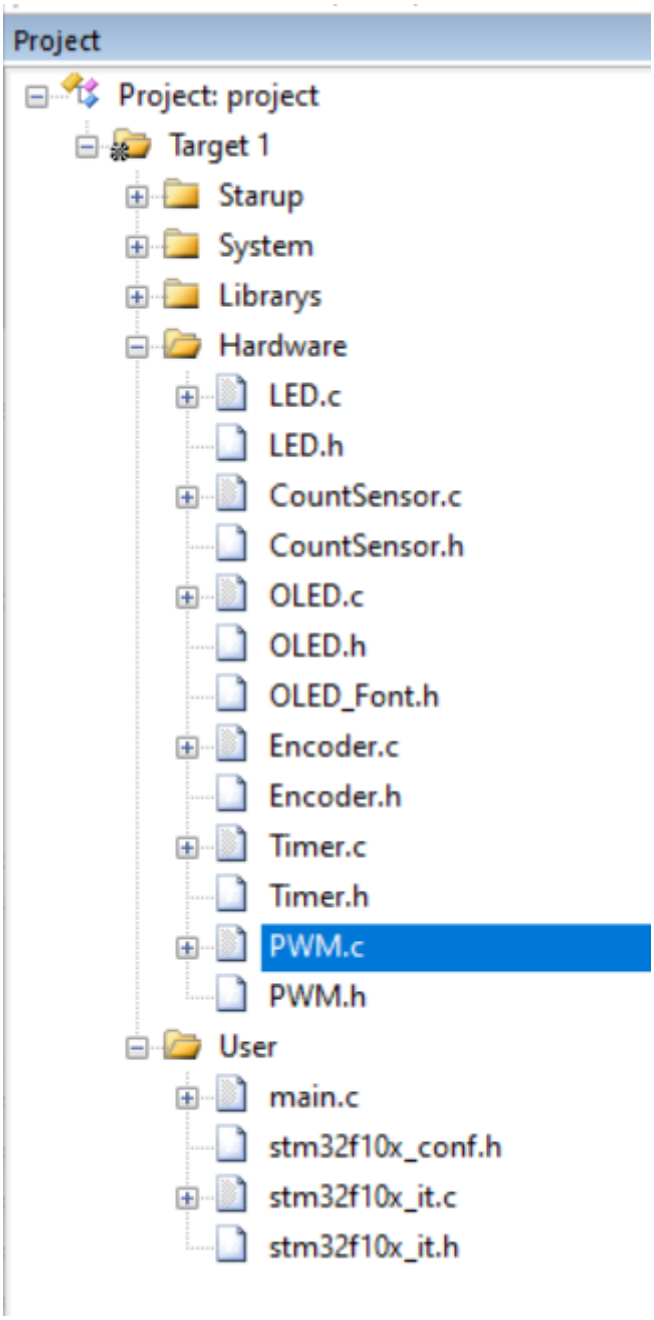
通过控制STM32输出指定PWM波形 1KHz，占空比50%，分辨率1% 的PWM波形。

（2）硬件电路接线图




（3）程序设计

工程结构如下图




具体代码如下

PWM.c

```
#include "stm32f10x.h"                  // Device header


//1000Hz 频率
void PWM_Init(void)
{
    //1.启动TIM2
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM2,ENABLE);
   //2.内部时钟源选择,如果没有设置就默认使用TIM2的时钟。
    TIM_InternalClockConfig(TIM2);
    //3.时基单元配置
    TIM_TimeBaseInitTypeDef TIM_TimeBaseInitTypeStructure;
    TIM_TimeBaseInitTypeStructure.TIM_ClockDivision = TIM_CKD_DIV1; //不分频，  此处无用
    TIM_TimeBaseInitTypeStructure.TIM_CounterMode = TIM_CounterMode_Up;//向上计数，还有向
下，中央对其
```

```
    TIM_TimeBaseInitTypeStructure.TIM_Period = 100-1; //ARR 自动重装计数器 ， 不能大于65535
    TIM_TimeBaseInitTypeStructure.TIM_Prescaler = 720-1; //PSC 预分频器 ， 不能大于65535
    TIM_TimeBaseInitTypeStructure.TIM_RepetitionCounter = 1; //重复计数，只有高级定时器才能设置
    TIM_TimeBaseInit(TIM2,&TIM_TimeBaseInitTypeStructure);
    //  按了复位键后从1开始计数的
    //  Generate an update event to reload the Prescaler and the Repetition counter values immediately
    TIM_ClearFlag(TIM2,TIM_FLAG_Update); //手动清除一次标识位
    //4.启动TIM2
    TIM_Cmd(TIM2,ENABLE);
    //5.输出比较配置
    TIM_OCInitTypeDef TIM_OCInitTypeStructure;
    //有的时候有很多参数用不上，但是又不能不设置==> 设置默认值
    TIM_OCStructInit(&TIM_OCInitTypeStructure); //设置默认值

    TIM_OCInitTypeStructure.TIM_OCMode = TIM_OCMode_PWM1; //模式PWM1选择
    TIM_OCInitTypeStructure.TIM_OCPolarity = TIM_OCPolarity_High; // 高电平 - 极性
    TIM_OCInitTypeStructure.TIM_OutputState = TIM_OutputState_Enable;//开启输出
    TIM_OCInitTypeStructure.TIM_Pulse = 50; //Capture Compare Register == CCR 不要大于ARR
    TIM_OC1Init(TIM2,&TIM_OCInitTypeStructure); //通道1配置

    //6.GPIO输出

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA,ENABLE);

    GPIO_InitTypeDef GPIO_InitTypeStructure;
    GPIO_InitTypeStructure.GPIO_Mode = GPIO_Mode_AF_PP; //复用推挽输出
    GPIO_InitTypeStructure.GPIO_Pin = GPIO_Pin_0 ; //TIM_OC1Init 对应的是PA0
    GPIO_InitTypeStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOA,&GPIO_InitTypeStructure);

}

//设置通道1的占空比
void PWM_SetCompare1(uint16_t compare)
{
    TIM_SetCompare1(TIM2,compare); //配置CCR的值，CCR 不要大于ARR
}
```

PWM.h

```
#ifndef __PWM_H
#define __PWM_H
void PWM_Init(void);
void PWM_SetCompare1(uint16_t compare);
#endif
```

main.c

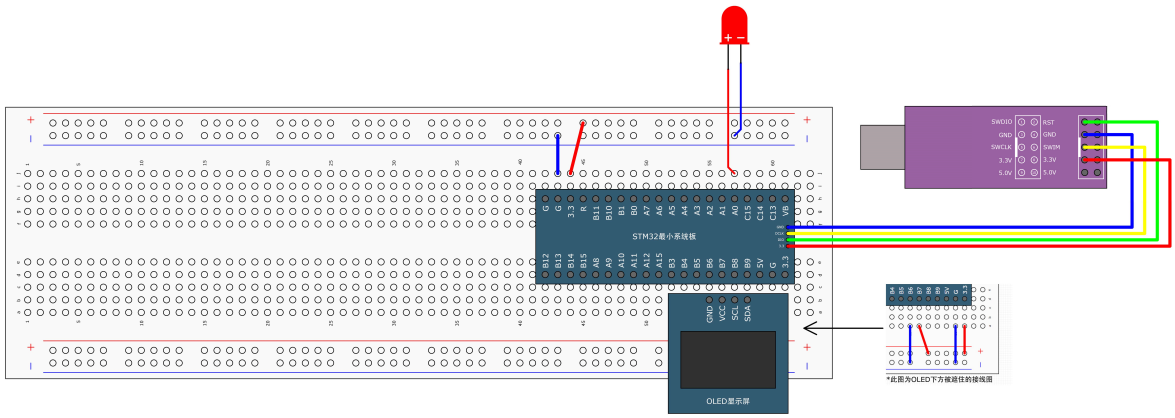
```c
#include "stm32f10x.h"                  // Device header
#include "OLED.h"
#include "PWM.h"


int main(void)
{

    OLED_Init();
    PWM_Init();
    PWM_SetCompare1(50);

    while(1)
    {

    }
}
```

## 2、PWM驱动LED呼吸灯

（1）实验目的

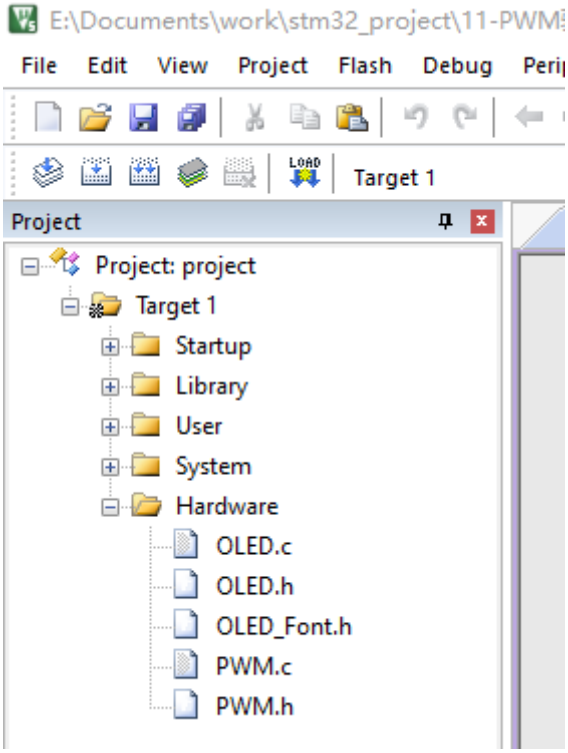
通过控制STM32输出PWM波形，调整波形的占空比从而实现LED等一闪一闪的呼吸灯效果。

（2）硬件电路接线图




（3）程序设计

工程结构如下图：




其中Hardware组与之前工程不一样，这个组主要增加了PWM相关驱动代码。具体代码展示如下：

PWM.c

```
#include "stm32f10x.h"                  // Device header

void PWM_Init(void) {
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM2, ENABLE);    // 打开TIM2时钟
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);   // 打开GPIOA时钟

    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;     // 配置复用推挽输出
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0;
    GPIO_Init(GPIOA, &GPIO_InitStructure);      // 初始化GPIOA

    TIM_InternalClockConfig(TIM2);      // 配置使用内部时钟

    TIM_TimeBaseInitTypeDef TIM_TimeBaseInitStructure;
    TIM_TimeBaseInitStructure.TIM_ClockDivision = TIM_CKD_DIV1;     // 配置滤波器时钟不分频
    TIM_TimeBaseInitStructure.TIM_CounterMode = TIM_CounterMode_Up;     // 配置向上计数
    TIM_TimeBaseInitStructure.TIM_Period = 100 - 1;     // 设置计数器ARR
    TIM_TimeBaseInitStructure.TIM_Prescaler = 720 - 1;      // 设置预分频器PSC
    TIM_TimeBaseInitStructure.TIM_RepetitionCounter = 0;    // 配置重复计数器，这里没用
    TIM_TimeBaseInit(TIM2, &TIM_TimeBaseInitStructure);     // 时基单元初始化

    TIM_OCInitTypeDef TIM_OCInitStructure;
    TIM_OCStructInit(&TIM_OCInitStructure);     // 输出比较结构体赋完整初值
    TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM1;   // 输出比较模式配置PWM1模式
    TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_High;   // 输出极性配置为高
    TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable;   // 输出使能
    // 期望信号频率1000Hz，占空比为50%，分辨率为1%，占空比Duty = CCR / (ARR + 1)
    // 频率 = 72m/(psc+1)/(arr+1)
```

```
    // 占空比 = ccr/(arr+1)
    // 分辨率 = 1/(arr+1)
    // 求解arr = 100, psc = 720, ccr = 50
    TIM_OCInitStructure.TIM_Pulse = 50; // CCR
    TIM_OC1Init(TIM2, &TIM_OCInitStructure);    // 配置TIM2的输出比较通道1初始化

    // 使能定时器
    TIM_Cmd(TIM2, ENABLE);
}

void PWM_SetCompare1(uint16_t Compare) {
    TIM_SetCompare1(TIM2, Compare);     // 设置CCR的值，CCR和ARR共同决定占空比
}
```

PWM.h

```
#ifndef __PWM_H
#define __PWM_H

void PWM_Init(void);
void PWM_SetCompare1(uint16_t Compare);

#endif
```

main.c

```
#include "stm32f10x.h"                  // Device header
#include "Delay.h"
#include "OLED.h"
#include "PWM.h"

int main(void){

    OLED_Init();
    PWM_Init();

    while(1){
        for (int i=0; i < 100; i++) {
            PWM_SetCompare1(i);     // 依次将定时器的CCR寄存器设置为0~100，PWM占空比逐渐增大，LED逐渐变亮
            Delay_ms(10);
        }
        for (int i=0; i < 100; i++) {
            PWM_SetCompare1(100 - i);   // 依次将定时器的CCR寄存器设置为100~0，PWM占空比逐渐减小，LED逐渐变暗
            Delay_ms(10);
        }
    }
}
```

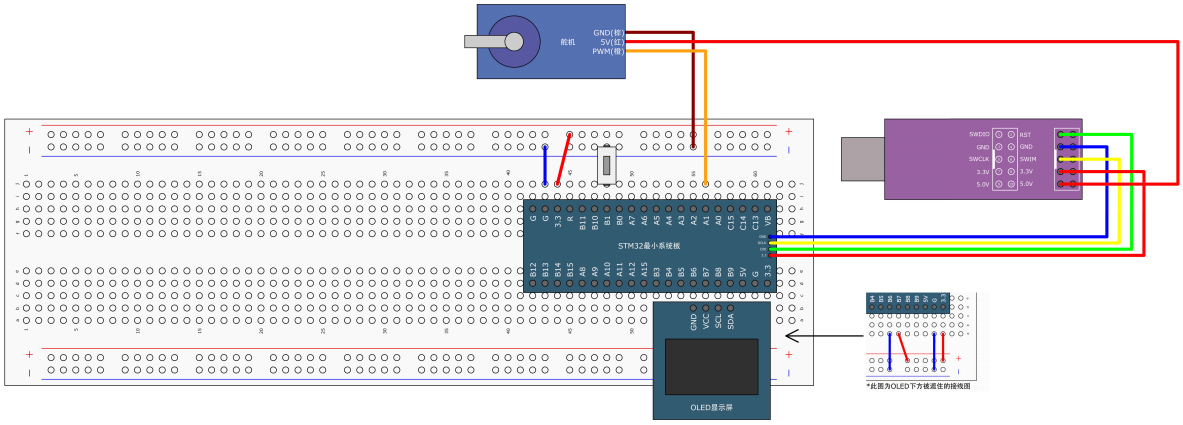
（4）代码完成后，编译并下载到硬件中执行，会发现实验结果是LED亮灭状态慢慢切换，符合预期设计。

### 3、PWM驱动舵机

（1）实验目的

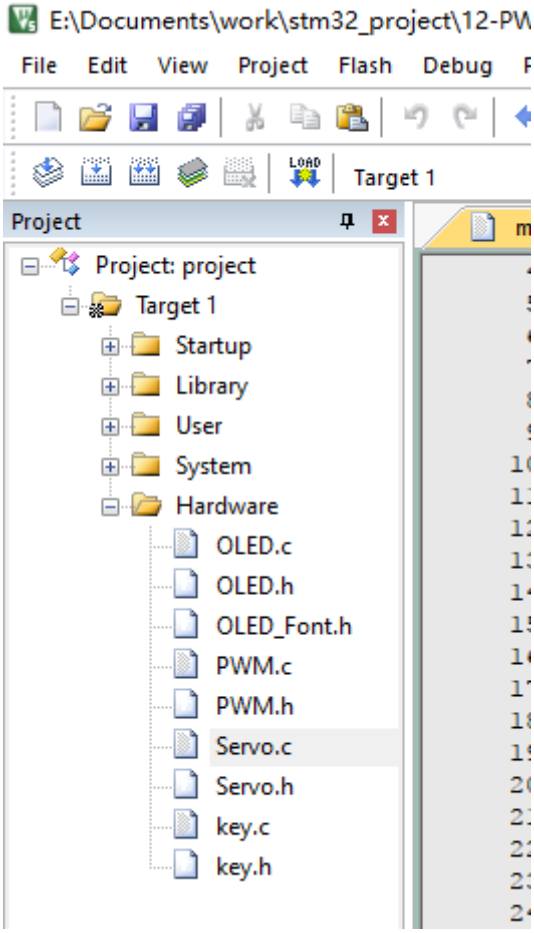
通过控制STM32输出PWM波形实现每按键一次，舵机旋转一个30度，多次按键可以控制舵机做180度旋转，超过180度后舵机将归位。

（2）硬件电路接线图



（3）程序设计

工程结构如下图：



其中Hardware组与之前工程相比，增加了舵机驱动和按键驱动，其中按键驱动代码同之前的按键项目，新增部分主要是舵机驱动，此外由于PWM波形是专为控制舵机调整的，所以PWM驱动中的ARR、PSC配置有所调整。具体代码展示如下：

Servo.c

```
#include "stm32f10x.h"                  // Device header
#include "PWM.h"

void Servo_Init(void) {
    PWM_Init();
}

void Servo_SetAngle(float angle) {
    PWM_SetCompare2(angle / 180 * 2000 + 500);  // 设置占空比，将角度变化转换为对CCR的设置
}
```

Servo.h

```
#ifndef __SERVO_H
#define __SERVO_H

void Servo_Init(void);
void Servo_SetAngle(float angle);

#endif
```

PWM.c

```
#include "stm32f10x.h"                  // Device header

void PWM_Init(void) {
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM2, ENABLE);
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);

    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_1;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    TIM_InternalClockConfig(TIM2);

    TIM_TimeBaseInitTypeDef TIM_TimeBaseInitStructure;
    TIM_TimeBaseInitStructure.TIM_ClockDivision = TIM_CKD_DIV1;
    TIM_TimeBaseInitStructure.TIM_CounterMode = TIM_CounterMode_Up;
    // 配置修改ARR、PSC的值，主要为了满足舵机控制50hz，高电平0.5ms~2.5ms变化的要求
    TIM_TimeBaseInitStructure.TIM_Period = 20000 - 1;       // ARR
    TIM_TimeBaseInitStructure.TIM_Prescaler = 72 - 1;       // PSC
    TIM_TimeBaseInitStructure.TIM_RepetitionCounter = 0;
```

```
    TIM_TimeBaseInit(TIM2, &TIM_TimeBaseInitStructure);

    TIM_OCInitTypeDef TIM_OCInitStructure;
    TIM_OCStructInit(&TIM_OCInitStructure);
    TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM1;
    TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_High;
    TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable;
    TIM_OCInitStructure.TIM_Pulse = 0;  // CCR
    TIM_OC2Init(TIM2, &TIM_OCInitStructure);

    // 使能定时器
    TIM_Cmd(TIM2, ENABLE);
}

void PWM_SetCompare2(uint16_t Compare) {
    TIM_SetCompare2(TIM2, Compare);
}
```

PWM.h

```
#ifndef __PWM_H
#define __PWM_H

void PWM_Init(void);
void PWM_SetCompare2(uint16_t Compare);

#endif
```

Key.c

```
#include "stm32f10x.h"                  // Device header
#include "Delay.h"

void KEY_Init(void) {
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB, ENABLE);
    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_1;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;

    GPIO_Init(GPIOB, &GPIO_InitStructure);
}

uint8_t KEY_GetNum(void) {
    uint8_t KeyNum = 0;
    if (GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_1) == 0) {
        Delay_ms(20);
        while (GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_1) == 0);
        Delay_ms(20);
```

```
        KeyNum = 1;
    }
    return KeyNum;
}
```

Key.h

```
# ifndef __KEY_H
# define __KEY_H

void KEY_Init(void);
uint8_t KEY_GetNum(void);

# endif
```

main.c

```
#include "stm32f10x.h"                  // Device header
#include "OLED.h"
#include "Servo.h"
#include "key.h"

uint8_t KeyNum;
float Angle;

int main(void){

    OLED_Init();
    Servo_Init();
    KEY_Init();
    OLED_ShowString(1, 1, "Angle:");

    while(1){
        KeyNum = KEY_GetNum();
        if (KeyNum == 1) {
            Angle += 30;    // 角度变量自增30
            if (Angle > 180) {  // 角度变量超过180后
                Angle = 0;  // 角度变量归零
            }
        }
        Servo_SetAngle(Angle);      // 设置舵机的角度为角度变量
        OLED_ShowNum(1, 7, Angle, 3);   // OLED显示角度变量
    }
}
```

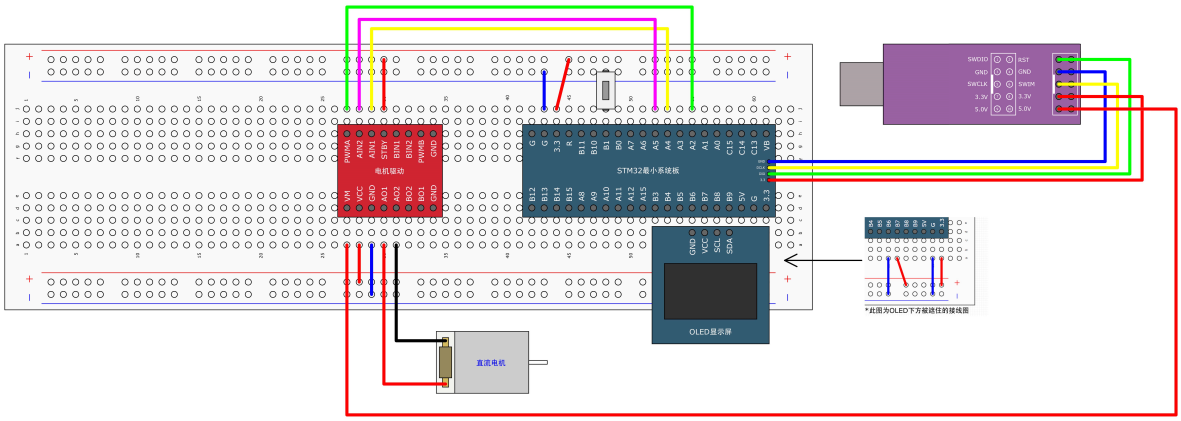
（4）代码完成后，编译并下载到硬件中执行，会发现实验结果是每按键一次，舵机就会对应旋转，符合预期设计。

### 4、PWM驱动直流电机

（1）实验目的

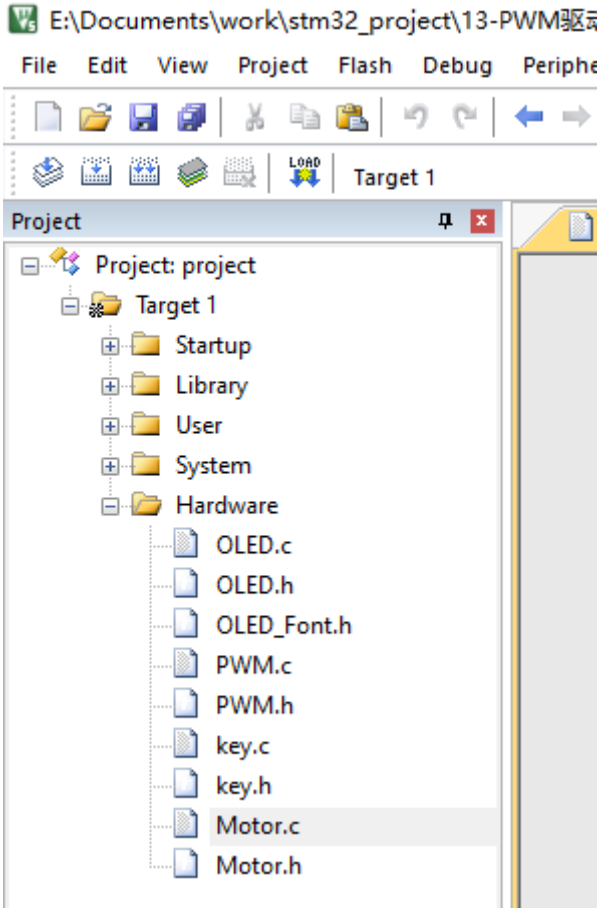
通过控制STM32输出PWM波形实现按键调整电机转速，每按一次电机转速增加，当转速超过100，再次按键转速自动归零。

（2）硬件电路接线图



（3）程序设计

工程结构如下图：




其中Hardware组与之前工程相比，增加了电机驱动，新增部分主要是电机驱动，此外由于PWM波形是专为控制电机转速调整的，所以PWM驱动中的ARR、PSC配置有所变化。具体代码展示如下：

Motor.c

```
#include "stm32f10x.h"                  // Device header
#include "PWM.h"
```

```
void Motor_Init(void) {
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);   // 开启GPIOA的时钟

    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_4 | GPIO_Pin_5;
    GPIO_Init(GPIOA, &GPIO_InitStructure);      // 将A4和A5引脚初始化为推挽输出

    PWM_Init();     // 初始化直流电机的底层PWM
}

void Motor_SetSpeed(int8_t speed) {
    if (speed >=0 ) {   // 如果设置正转的速度值
        GPIO_SetBits(GPIOA, GPIO_Pin_4);    // A4引脚置高电平
        GPIO_ResetBits(GPIOA, GPIO_Pin_5);  // A5引脚置低电平，设置方向为正转
        PWM_SetCompare3(speed);     // PWM设置为速度值
    } else {    // 否则设置反转的速度值
        GPIO_ResetBits(GPIOA, GPIO_Pin_4);  // A4引脚置低电平
        GPIO_SetBits(GPIOA, GPIO_Pin_5);    // A5引脚置高电平，设置方向为反转
        PWM_SetCompare3(-speed);    // PWM设置为负的速度值，因为此时速度值为负数，而PWM只能给
正数
    }
}
```

Motor.h

```
#ifndef __MOTOR_H
#define __MOTOR_H

void Motor_Init(void);
void Motor_SetSpeed(int8_t speed);

#endif
```

PWM.c

```
#include "stm32f10x.h"                  // Device header

void PWM_Init(void) {
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM2, ENABLE);
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);

    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_2;
    GPIO_Init(GPIOA, &GPIO_InitStructure);
```

```
    TIM_InternalClockConfig(TIM2);

    TIM_TimeBaseInitTypeDef TIM_TimeBaseInitStructure;
    TIM_TimeBaseInitStructure.TIM_ClockDivision = TIM_CKD_DIV1;
    TIM_TimeBaseInitStructure.TIM_CounterMode = TIM_CounterMode_Up;
    // 为了减少点击噪声，通过配置ARR和PSC将PWM输出频率设置为20k，人耳听力上限。
    TIM_TimeBaseInitStructure.TIM_Period = 100 - 1;     // ARR
    TIM_TimeBaseInitStructure.TIM_Prescaler = 36 - 1;       // PSC
    TIM_TimeBaseInitStructure.TIM_RepetitionCounter = 0;
    TIM_TimeBaseInit(TIM2, &TIM_TimeBaseInitStructure);

    TIM_OCInitTypeDef TIM_OCInitStructure;
    TIM_OCStructInit(&TIM_OCInitStructure);
    TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM1;
    TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_High;
    TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable;
    TIM_OCInitStructure.TIM_Pulse = 0;  // CCR
    TIM_OC3Init(TIM2, &TIM_OCInitStructure);

    // 使能定时器
    TIM_Cmd(TIM2, ENABLE);
}

void PWM_SetCompare3(uint16_t Compare) {
    TIM_SetCompare3(TIM2, Compare);
}
```

PWM.h

```
#ifndef __PWM_H
#define __PWM_H

void PWM_Init(void);
void PWM_SetCompare3(uint16_t Compare);

#endif
```

main.c

```
#include "stm32f10x.h"                  // Device header
#include "OLED.h"
#include "Motor.h"
#include "key.h"

uint8_t KeyNum;
int8_t Speed;

int main(void){
```

```
    OLED_Init();
    Motor_Init();
    KEY_Init();
    OLED_ShowString(1, 1, "Speed:");

    while(1){
        KeyNum = KEY_GetNum();
        if (KeyNum == 1) {
            Speed += 20;    // 速度变量自增20
            if (Speed > 100) {  // 速度变量超过100后
                Speed = -100;   // 速度变量变为-100。此操作会让电机旋转方向突然改变，可能会因供电不足而导致单片机复位
                                // 若出现了此现象，则应避免使用这样的操作
            }
        }
        Motor_SetSpeed(Speed);
        OLED_ShowSignedNum(1, 7, Speed, 3);
    }
}
```

（4）代码完成后，编译并下载到硬件中执行，会发现实验结果是每按键一次，电机转速都会对应变化，符合预期设计。

## 课程小结

1. TIM包含基本、通用、高级三种类型的定时器，可以通过定时器的计数并申请中断，配合系统完成各种时钟控制功能。无论哪种定时器时基单元的运作原理都是其中的核心，需要重点理解ARR、PSC、CNT的关系。
2. OC依然与各种定时器电路密切相关，只不过它的核心重点是时基单元和其后的输出比较部分，通过CNT和CCR的不断比较，从而输出电平不断变化的PWM波形。利用ARR、PSC以及CCR配置值的不同，就可以动态改变PWM波形的频率以及占空比。利用这样的波形最终对外设进行控制驱动。

## 随堂作业

1. 请说说EXTI和定时中断有什么区别。
2. 请说明现在需要一个5Hz频率，占空比为30的PWM波形，该如何设置ARR、PSC以及CCR呢?
3. 设计两个按键，可以增加或减少输出的频率，频率用示波器查看。
4. 根据环境温度，温度越高风扇越快